```c
#include <stdio.h>

#define SIZE 10   // m = number of memory locations

int hashTable[SIZE];

// Initialize hash table
void init() {
   int i;
   for (i = 0; i < SIZE; i++)
      hashTable[i] = -1;
}

// Insert key using linear probing
void insert(int key) {
   int index = key % SIZE;
   int startIndex = index;

   while (hashTable[index] != -1) {
      index = (index + 1) % SIZE;

      if (index == startIndex) {
         printf("Hash table is full\n");
         return;
      }
   }

   hashTable[index] = key;
   printf("Key %d inserted at address %d\n", key, index);
}
```

```
// Display hash table
void display() {
   int i;
   printf("\nHash Table:\n");
   for (i = 0; i < SIZE; i++) {
      if (hashTable[i] != -1)
         printf("Address %d : %d\n", i, hashTable[i]);
      else
         printf("Address %d : EMPTY\n", i);
   }
}

int main() {
   int n, key, i;

   init();

   printf("Enter number of employee records: ");
   scanf("%d", &n);

   for (i = 0; i < n; i++) {
      printf("Enter 4-digit employee key: ");
      scanf("%d", &key);
      insert(key);
   }

   display();

   return 0;
}
```

```
Enter number of employee records: 3
Enter 4-digit employee key: 234
Key 234 inserted at address 4
Enter 4-digit employee key: 1244
Key 1244 inserted at address 5
Enter 4-digit employee key: 1254
Key 1254 inserted at address 6

Hash Table:
Address 0 : EMPTY
Address 1 : EMPTY
Address 2 : EMPTY
Address 3 : EMPTY
Address 4 : 234
Address 5 : 1244
Address 6 : 1254
Address 7 : EMPTY
Address 8 : EMPTY
Address 9 : EMPTY

Process returned 0 (0x0)   execution time : 7.165 s
Press any key to continue.
```